

Discovery Executive Summary

Project: test-disocvery · Generated: 18/08/2026, 17:33:05

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with a Laravel 12 backend (6 API controllers, 4 Eloquent models, 2 services), a React 19 SPA frontend (4 page components, 4 shared components, 1 custom hook, 1 Zustand store), and a Node/Express dev-API (6 source files). Backend controllers are lean on average (74 LOC) but 4 of 6 access Eloquent models directly — there is no repository layer, and business logic (reachability calculations, tree building) is duplicated across controllers and services in 3 independent locations. The `LegacyReportController` crosses both Discovery and Connect domain boundaries, reading all four models, while `DashboardController` does the same for aggregation. The `LegacyDataMapper` uses PHP `extract()` — a dynamic-variable anti-pattern that creates implicit coupling. On the frontend, all 4 page components and 1 legacy class component make direct `api.*` calls with no service/data-access abstraction layer; the `LegacyMonitorPoller` is a class component with an interval memory leak. The two most urgent risks are (1) the missing repository pattern which scatters ORM access across 32 call sites, and (2) the cross-domain coupling in `LegacyReportController` and `DashboardController` that prevents independent evolution of Discovery and Connect modules. Layers covered: backend (26 PHP files), frontend (15 TS/TSX/JSX files), dev-api (6 JS files).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg (max 102)	Good
H2	Missing Service Layer	Controllers accessing repos/models	<15	15–30	>30	25 direct model calls in controllers	Moderate
H3	Missing Repository Pattern	Direct DB access points outside repos	<15	15–30	>30	32 (25 controllers + 7 services)	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (LegacyDataMapper)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (all Eloquent, no raw SQL)	Good
H7	God Classes	Largest class/file LOC	<500	500–600	>600	276 LOC (dev-api server.js); 222 LOC (ConnectPage.tsx); 165 LOC (MongoService.php)	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (LegacyReportController: 4 cross-domain models; DashboardController: 2)	Moderate
H9	Shared Database Coupling	Tables shared across domains	<30%	30–40%	>40%	~33% — 2 of 4 models queried by cross-domain controllers	Moderate
H10	Duplicated Business Logic (additional)	Independent copies of same algorithm	0	1–2	>2	3 copies of reachability formula + 3 copies of buildTree	Moderate
H11	Missing Frontend Service Layer	Page components with direct API	0	1–3	>3	5 components with 14 total direct api.* calls	Moderate

	(additional)	calls					
--	--------------	-------	--	--	--	--	--

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Introduce <code>ConnectMonitorRepository</code> and <code>DiscoveryJobRepository</code> interfaces with Eloquent implementations; bind in <code>AppServiceProvider</code> ; replace all 32 direct <code>Model::</code> calls	High Risk	High
H2. Missing Service Layer	Create <code>ConnectReachabilityService</code> , <code>DiscoveryTreeService</code> , <code>DashboardKpiService</code> ; move business logic out of 4 controllers	Moderate	High
H10. Duplicated Business Logic	Consolidate 3 copies of reachability formula and 3 copies of <code>buildTree()</code> into single canonical service methods	Moderate	High
H8. Domain Boundary Violations	Split <code>LegacyReportController</code> by domain; route cross-domain reads through service interfaces	Moderate	Medium
H9. Shared Database Coupling	Define table ownership per domain; expose read-only service APIs for cross-domain consumers	Moderate	Medium
H5. Shared Utility Abuse	Remove <code>extract()</code> from <code>LegacyDataMapper</code> and <code>LegacyReportController</code> ; move mapping logic to domain service	Moderate	Medium
H11. Missing Frontend Service Layer	Create <code>connectService.ts</code> / <code>discoveryService.ts</code> ; extract domain hooks; convert <code>LegacyMonitorPoller</code> to functional component with cleanup	Moderate	Medium

1.5 Expected Outcomes

- Testability:** Repository interfaces enable unit testing of services without a database; frontend service layer enables mocking API calls in component tests.
 - Single source of truth:** Consolidating the reachability formula and `buildTree()` into canonical service methods eliminates the risk of divergent business logic across endpoints.
 - Independent module evolution:** Enforcing domain boundaries (Discovery vs. Connect) through service interfaces means schema changes in one domain cannot silently break the other.
 - Onboarding velocity:** New contributors can work on Discovery or Connect independently without needing to understand both domains — bounded contexts reduce the knowledge surface area.
 - Migration readiness:** Repository abstraction + domain services position the codebase for future extraction into separate deployable services, should the platform scale beyond a monolith.
- ```

{
 "stop_reason": "end_turn",
 "session_id": "a718565e-0393-476a-8617-856b70cd46ec",
 "total_cost_usd": 2.3541065,
 "usage": {
 "input_tokens": 18,
 "cache_creation_input_tokens": 98918,
 "cache_read_input_tokens": 1147903,
 "output_tokens": 31228,
 "server_tool_use": {
 "web_search_requests": 0,
 "web_fetch_requests": 0,
 "service_tier": "standard",
 "cache_creation": {
 "ephemeral_1h_input_tokens": 98918,
 "ephemeral_5m_input_tokens": 0,
 "inference_geo": "not_available",
 "iterations": [
 {
 "input_tokens": 1,
 "output_tokens": 1999,
 "cache_read_input_tokens": 106182,
 "cache_creation_input_tokens": 10409,
 "cache_creation": {
 "ephemeral_5m_input_tokens": 0,
 "ephemeral_1h_input_tokens": 10409,
 "type": "message"
 }
 }
],
 "speed": "standard",
 "modelUsage": {
 "claude-haiku-4-5-20251001": {
 "inputTokens": 10105,
 "outputTokens": 16,
 "cacheReadInputTokens": 0,
 "cacheCreationInputTokens": 0,
 "webSearchRequests": 0,
 "costUSD": 0.010185,
 "contextWindow": 200000,
 "maxOutputTokens": 32000,
 "opus-4-6": {
 "inputTokens": 18,
 "outputTokens": 31228,
 "cacheReadInputTokens": 1147903,
 "cacheCreationInputTokens": 98918,
 "webSearchRequests": 0,
 "costUSD": 2.3439215,
 "contextWindow": 200000,
 "maxOutput": [],
 "terminal_reason": "completed",
 "fast_mode_state": "off",
 "uuid": "f330f0c5-bea8-4977-88b2-fd4b9b4d5a8e"
 }
 }
 }
 }
 }
 }
}
```